Cátedra Universitaria: De Cero a Maestro en Git y GitHub

Guía Pedagógica y Material Docente para Clase Magistral
Área: Fundamentos de Ingeniería de Software — Universidad de la Amazonia
Diseñado para: Exposición y Enseñanza desde Cero Absoluto (Enfoque Universal e Inclusivo)
Objetivo: Explicar qué es el control de versiones con analogías cotidianas (cocina, redacción, fotografía), cómo funciona Git internamente, qué aporta GitHub y por qué es el estándar universal de la industria.

📑 Índice General de la Cátedra

1. **Módulo 0: El Punto de Partida — El Miedo Humano a Equivocarse y Perder Trabajo**
2. **1.1 La pesadilla cotidiana de los archivos duplicados**
3. **1.2 La analogía del 'Ctrl+Z' con Memoria Infinita y las Recetas de Cocina**
4. **1.3 ¿Qué pasa cuando varias personas editan el mismo documento?**
5. **Módulo 1: ¿Qué es exactamente Git? (El Motor Local)**
6. **2.1 Historia y Origen: Linus Torvalds y la necesidad de colaborar**
7. **2.2 Definición Formal: Sistema de Control de Versiones Distribuido (DVCS)**
8. **2.3 ¿Cómo ve Git los archivos? Instantáneas (*Snapshots*) y Huellas Digitales**
9. **Módulo 2: ¿Qué es exactamente GitHub? (La Red de Colaboración)**
10. **3.1 Si ya tengo Git en mi PC, ¿para qué necesito GitHub?**
11. **3.2 Las Grandes Analogías: Motor vs Autopista | Grabación vs Plataforma**
12. **3.3 Tabla Comparativa Definitiva: Git vs. GitHub**
13. **Módulo 3: El Modelo Mental de Git — Las 3 Zonas Locales + La Nube**
14. **4.1 La analogía del Estudio Fotográfico y el Álbum Familiar**
15. **4.2 Zona 1: Working Directory (El Escritorio de Trabajo)**
16. **4.3 Zona 2: Staging Area (La Mesa de Preparación)**
17. **4.4 Zona 3: Local Repository (El Álbum de Fotos Inmutable)**
18. **4.5 Zona 4: Remote Repository (La Galería en la Nube - GitHub)**
19. **Módulo 4: Guía Práctica de Comandos (Paso a Paso desde Cero)**
20. **5.1 Identificarse ante Git (`git config`)**
21. **5.2 Crear o Traer un Proyecto (`git init` y `git clone`)**
22. **5.3 La Rutina Diaria: `status`, `add` y `commit`**
23. **5.4 Ramas (Branches): La Analogía del Tazón de Prueba y Bocetos Paralelos**
24. **5.5 Unir Caminos: `merge` y la Resolución de Conflictos**
25. **5.6 Sincronizar con la Nube: `remote`, `push`, `fetch` y `pull`**
26. **5.7 Deshacer Errores y Auditoría (`log`, `restore`, `stash`, `diff`)**
27. **Módulo 5: El Ecosistema GitHub — Cómo Trabajan los Equipos Reales**
28. **6.1 Anatomía de un Repositorio: README, .gitignore y Licencias**
29. **6.2 Pull Requests (PR): La Revisión de Código por Pares**
30. **6.3 Gestión de Proyectos: Issues, Milestones y Tableros Kanban**
31. **6.4 Automatización de Calidad: GitHub Actions (CI/CD)**
32. **Módulo 6: ¿Por qué es el Corazón de la Hoja de Vida de un Ingeniero?**
33. **7.1 El fin del currículum en papel: "Proof of Work" (Prueba de Trabajo)**

34. **7.2 Qué busca un reclutador técnico en tu perfil**
35. **7.3 Buenas prácticas para un perfil profesional impecable**
36. **Módulo 7: Guion de Exposición para la Clase (Estructura Pedagógica)**
37. **8.1 Cronograma sugerido (20 minutos)**
38. **8.2 Dinámicas de inicio para conectar con todo el salón**
39. **8.3 Preguntas del profesor y respuestas de nivel senior**
40. **Módulo 8: Directorio de Enlaces y Recursos de Estudio**

---

1. Módulo 0: El Punto de Partida — El Miedo Humano a Equivocarse y Perder Trabajo

Para comprender por qué existen Git y GitHub, primero debemos observar un comportamiento humano universal: **el miedo a perder lo que ya construimos cuando queremos probar algo nuevo**.

1.1 La pesadilla cotidiana de los archivos duplicados

Cualquier persona que haya redactado un informe grupal, una tesis, un diseño o un proyecto universitario ha vivido esta escena:

```
📁 Carpeta_Proyecto/
   ├── Documento_v1.docx
   ├── Documento_v2_con_correcciones.docx
   ├── Documento_v2_con_aportes_de_Laura.docx
   ├── Documento_v3_final.docx
   ├── Documento_v3_final_FINAL.docx
   ├── Documento_v3_final_este_si_es_el_bueno.docx
   └── Documento_v3_final_imprimir_ESTE_POR_FAVOR.docx
```

¿Por qué hacemos esto?

No es por falta de orden, sino por **instinto de preservación**: * Queremos cambiar un párrafo o agregar una sección arriesgada, pero nos da pánico borrar lo anterior por si nos arrepentimos. * Duplicamos el archivo como "red de seguridad". * Al cabo de tres semanas, el equipo no sabe cuál es el archivo real, qué cambios hizo cada persona ni cuál es la versión válida.

1.2 La analogía del 'Ctrl+Z' con Memoria Infinita y las Recetas de Cocina

Para entender el valor de Git sin tecnicismos, pensemos en dos situaciones cotidianas:

1. La Receta de Repostería (La Masa Base):

- Imagina que estás preparando una masa pastelera perfecta. Tienes la base lista y quieres experimentar agregándole ralladura de naranja o esencia de café.
- Si le echas el ingrediente directamente a toda la masa y no sabe bien, **arruinaste todo el pastel y perdiste horas de trabajo**.
- ¿Qué hace un cocinero inteligente? **Toma una pequeña porción en un tazón separado (*rama*) y experimenta allí**.
- Si el sabor es espectacular, lo integra con orgullo a la masa principal (*merge*). Si sabe mal, simplemente descarta ese tazón pequeño y la masa base sigue perfecta e intacta.

2. El 'Ctrl+Z' con Memoria Eterna:

- En Word o Photoshop puedes presionar `Ctrl + Z` para deshacer un error, pero si guardas y cierras el programa, ese historial se borra para siempre.

- **Git es un `Ctrl + Z` indestructible:** Te permite viajar en el tiempo a cómo estaba tu documento hace 5 minutos, hace tres semanas o hace un año, con la seguridad absoluta de que nada se pierde jamás.

1.3 ¿Qué pasa cuando varias personas editan el mismo documento?

Imaginemos que 4 integrantes de un equipo deben redactar una propuesta juntos:

1. **Sin Git (El método caótico):**
2. Valentina abre el archivo a las 2:00 PM y redacta la introducción.
3. Camilo abre el mismo archivo a las 2:05 PM y redacta los objetivos.
4. Valentina guarda su trabajo a las 2:30 PM.
5. Camilo guarda su trabajo a las 2:31 PM.
6. **Resultado:** El guardado de Camilo **sobrescribe y destruye** todo lo que Valentina escribió. Valentina perdió su tarde y se genera un conflicto de equipo.
7. **Con Git (El método profesional):**
8. Git compara los documentos **línea por línea**.
9. Si Valentina modificó los primeros párrafos y Camilo los del final, Git **combina automáticamente** ambos aportes sin que nadie pierda una sola palabra.
10. Si ambos modificaron la misma frase exacta, Git frena la operación con cortesía, muestra ambas propuestas y les pide acordar cuál dejar. Nadie es silenciado ni sobrescrito.

---

2. Módulo 1: ¿Qué es exactamente Git? (El Motor Local)

2.1 Historia y Origen: Linus Torvalds y la necesidad de colaborar

En el año 2005, miles de ingenieros alrededor del mundo colaboraban en el desarrollo del sistema operativo Linux. La herramienta que usaban para coordinarse dejó de ser gratuita, y las demás opciones del mercado eran desesperadamente lentas.

Linus Torvalds (el creador de Linux) decidió tomarse un receso y programar una herramienta basada en velocidad, integridad de datos y autonomía para cada usuario. La bautizó **Git**.

2.2 Definición Formal: Sistema de Control de Versiones Distribuido (DVCS)

- **Sistema de Control de Versiones:** Un software que registra y organiza la historia completa de un conjunto de archivos.
- **Distribuido (*Distributed*):** Significa que **cada persona del equipo tiene en su computador una copia exacta, completa e idéntica de toda la historia del proyecto**.
- Si se cae la red de la universidad, si se apaga el servidor central o si estás trabajando en una zona rural sin cobertura, puedes seguir trabajando, creando versiones y consultando el pasado con total normalidad.

2.3 ¿Cómo ve Git los archivos? Instantáneas (*Snapshots*) y Huellas Digitales

A diferencia de los sistemas antiguos que guardaban listas complejas de "diferencias sueltas", Git piensa en términos de **fotografías completas (*snapshots*)**.

- Cada vez que guardas una versión (*commit*), Git toma una "foto" de cómo lucen todos tus archivos en ese instante.
- Si un archivo no cambió entre una versión y otra, Git no gasta espacio duplicándolo; simplemente apunta a la foto anterior.

- Cada foto se identifica con una huella digital matemática inmutable llamada **Hash SHA-1** (ejemplo: `68b6d7f52ce...`). Esto garantiza que nadie pueda alterar o corromper una versión del pasado sin que el sistema lo detecte inmediatamente.

---

3. Módulo 2: ¿Qué es exactamente GitHub? (La Red de Colaboración)

3.1 Si ya tengo Git en mi PC, ¿para qué necesito GitHub?

Git es una herramienta que vive dentro de su propia computadora (en la terminal). Sin embargo, mientras su computador esté apagado o desconectado, nadie más puede acceder a su trabajo.

GitHub es una plataforma en la nube que actúa como el **punto de encuentro global**: es el lugar seguro en internet donde subimos nuestros repositorios Git para respaldarlos, compartirlos con colegas, recibir revisiones y conectar con empresas de todo el mundo.

3.2 Las Grandes Analogías: Motor vs Autopista | Grabación vs Plataforma

Para despejar cualquier confusión en el aula:

- **Analogía del Automóvil:**
- **Git** es el **motor**: la ingeniería interna que hace que el carro avance y funcione mecánicamente.
- **GitHub** es la **autopista y el club de conductores**: el espacio compartido donde interactúas con otros, muestras tu vehículo y viajas en comunidad.
- **Analogía de la Música:**
- **Git** es el **estudio de grabación**: donde grabas las pistas, corriges notas y guardas tus masters.
- **GitHub** es **Spotify**: donde publicas tus álbumes para que el mundo los escuche y colabore contigo.

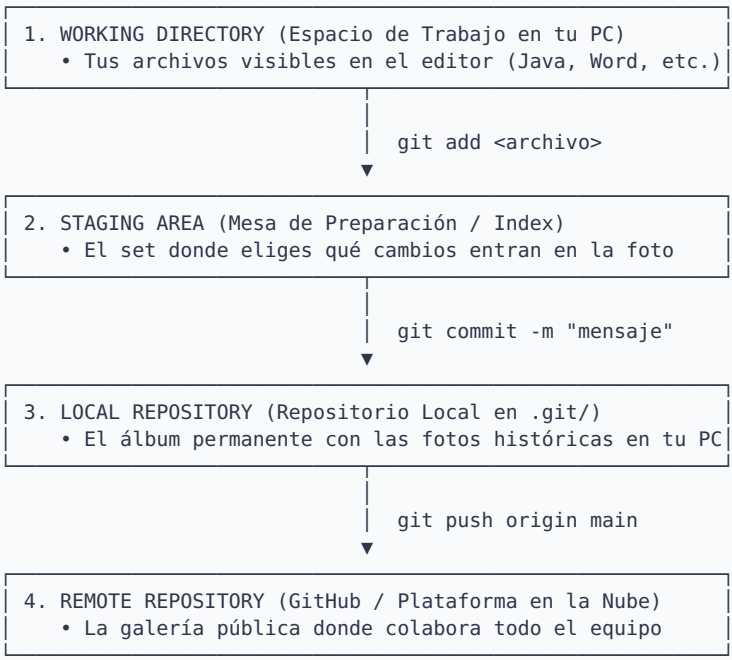
3.3 Tabla Comparativa Definitiva: Git vs. GitHub

Criterio	Git	GitHub
¿Qué es?	Software / Herramienta de línea de comandos.	Plataforma web / Servicio en la nube (SaaS).
¿Dónde se ejecuta?	100% en local en tu computadora.	En los servidores seguros de Microsoft/GitHub.
¿Requiere Internet?	**NO.** Funciona sin señal en cualquier lugar.	**SÍ.** Para sincronizar, navegar o revisar PRs.
Interfaz	Consola de comandos (CLI) o visualizadores.	Interfaz web interactiva y amigable.
Costo	Gratuito y de código abierto (Open Source).	Gratuito con planes empresariales avanzados.
Alternativas	Mercurial, Subversion (SVN).	GitLab, Bitbucket, Codeberg.

---

4. Módulo 3: El Modelo Mental de Git — Las 3 Zonas Locales + La Nube

Este esquema resume la estructura operativa de Git:



4.1 La analogía del Estudio Fotográfico y el Álbum Familiar

Imaginemos que eres fotógrafo en un evento social:

1. **Working Directory (El salón del evento):** Hay 50 invitados caminando y conversando (tus archivos en edición).
2. **Staging Area (El set de fotografía):** Dices: "*Por favor, que pasen solo los graduados al frente de la cámara*" (`git add`). Los demás invitados permanecen en el salón.
3. **Local Repository (El disparo de la cámara y el álbum físico):** Haces *¡CLIC!* (`git commit -m "Foto oficial de graduados"`). Esa fotografía queda impresa y encuadernada en el álbum de tu casa para siempre.
4. **Remote Repository (Publicar la foto en la galería web):** Subes esa fotografía a GitHub para que toda la familia y compañeros puedan verla y descargarla (`git push`).

4.2 Zona 1: Working Directory (El Escritorio de Trabajo)

Es la carpeta visible en su sistema operativo. Aquí es donde usted crea archivos, programa clases en Java (`Persona.java` , `Motorista.java`), borra líneas y experimenta. Estos cambios son provisionales.

4.3 Zona 2: Staging Area (La Mesa de Preparación)

Es el espacio intermedio (el *Index*). Le permite a usted seleccionar con precisión quirúrgica qué archivos y qué cambios formarán parte de la próxima versión, permitiendo crear entregas limpias y ordenadas.

4.4 Zona 3: Local Repository (El Álbum Inmutable)

Es la base de datos interna dentro de la carpeta oculta `.git/` . Al hacer un commit, se almacena una instantánea definitiva con su nombre, correo, fecha y mensaje explicativo.

4.5 Zona 4: Remote Repository (La Galería en la Nube - GitHub)

Es la réplica alojada en los servidores de GitHub, que permite coordinar el trabajo en equipo y respaldar el proyecto contra fallos de hardware.

---

5. Módulo 4: Guía Práctica de Comandos (Paso a Paso desde Cero)

5.1 Identificarse ante Git (`git config`)

Al instalar Git por primera vez, es necesario registrar la firma que acompañará a cada uno de sus aportes:

```
# Registrar nombre y apellido
git config --global user.name "Gian Marco Castañeda"

# Registrar correo institucional
git config --global user.email "marco@uniamazonia.edu.co"

# Establecer 'main' como el nombre estándar de la rama principal
git config --global init.defaultBranch main

# Verificar los datos configurados
git config --list
```

5.2 Crear o Traer un Proyecto (`git init` y `git clone`)

- **Opción A: Iniciar el control de versiones en una carpeta existente:** `bash cd /mi/carpeta/proyecto git init` *(Crea la carpeta interna `.git/` y comienza a monitorear los archivos).*
- **Opción B: Clonar un proyecto que ya está en GitHub a su computadora:** `bash git clone https://github.com/Stellel-One/TucanGo.git` *(Descarga todo el código, ramas y el historial completo a su disco local).*

5.3 La Rutina Diaria: `status`, `add` y `commit`

El flujo de trabajo profesional diario consiste en tres pasos fundamentales:

```
# 1. Inspeccionar el estado de los archivos (modificados, nuevos o preparados)
git status

# 2. Agregar un archivo específico a la mesa de preparación (Staging Area)
git add src/co/edu/uniamazonia/logica2/modelo/Persona.java

# O agregar todos los cambios de una sola vez
git add .

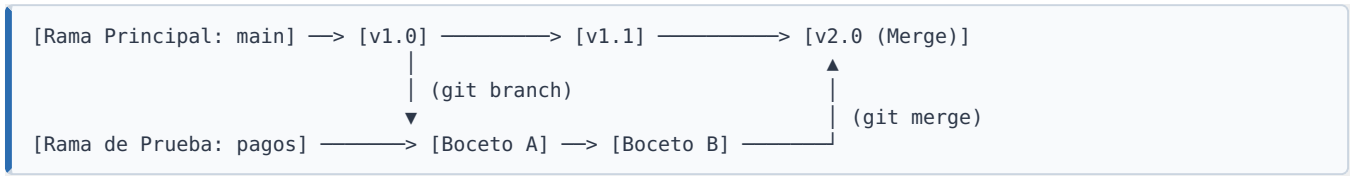
# 3. Guardar la versión permanente con un mensaje claro
git commit -m "feat(modelo): crea clase base Persona y entidad Estudiante"
```

> 📌 **Convenciones Profesionales para Mensajes de Commit:** * `feat:` Incorporación de una nueva funcionalidad. * `fix:` Corrección de un error o falla en el código. * `docs:` Modificaciones exclusivas en documentación (`README.md` , guías). * `refactor:` Mejoras en la estructura del código sin alterar su funcionamiento.

5.4 Ramas (Branches): La Analogía del Tazón de Prueba y Bocetos Paralelos

Imagine que está diseñando el plano de una casa o redactando un libro:

- La rama `main` es el manuscrito principal y aprobado.
- Si desea redactar un final alternativo o probar un nuevo diseño de fachada, **no borra el original**: crea un **boceto paralelo (*rama*)**:




```
# Consultar las ramas existentes (* marca la rama donde estás ubicado)
git branch

# Crear una rama nueva para una función específica
git branch feat/modulo-pagos

# Cambiarse a la nueva rama para trabajar en ella
git switch feat/modulo-pagos

# Atajo profesional: Crear la rama y posicionarse en ella en un solo comando
git switch -c feat/modulo-pagos
```

Cualquier cambio realizado dentro de `feat/modulo-pagos` queda completamente aislado y **no altera la rama principal**. Si el experimento no funciona, la rama se descarta sin consecuencias para el proyecto estable.

5.5 Unir Caminos: `merge` y la Resolución de Conflictos

Cuando la funcionalidad desarrollada en la rama paralela ha sido probada y aprobada, se incorpora a la rama principal:

```
# 1. Posicionarse en la rama receptora (usualmente main)
git switch main

# 2. Fusionar los cambios de la rama secundaria
git merge feat/modulo-pagos
```

¿Qué es un Conflicto de Integración?

Se presenta cuando dos desarrolladores modifican exactamente la misma línea de un archivo con contenidos distintos. Git pausa la operación y presenta ambas propuestas:

```
<<<<<<< HEAD (Versión actual en main)
    tarifa = 4500.0;
=======
    tarifa = 4000.0;
>>>>>>> feat/modulo-pagos (Propuesta de la rama)
```

Resolución: El equipo dialoga, edita el archivo conservando el valor acordado, elimina los marcadores (`<<<<`, `====`, `>>>>`) y registra un commit de consolidación.

5.6 Sincronizar con la Nube: `remote`, `push`, `fetch` y `pull`

```
# Consultar los repositorios remotos vinculados
git remote -v

# Enviar los commits locales a la nube (rama main en GitHub)
git push origin main

# Descargar novedades del equipo desde GitHub SIN mezclarlas de inmediato
git fetch origin

# Descargar e integrar automáticamente las novedades del equipo en su equipo
git pull origin main
```

5.7 Deshacer Errores y Auditoría (`log`, `restore`, `stash`, `diff`)

- `git log --oneline --graph`: Despliega el árbol histórico de cambios de forma visual y resumida.
- `git diff`: Muestra con precisión las líneas eliminadas (rojo `-`) y agregadas (verde `+`).
- `git restore archivo.java`: Descarta modificaciones locales no guardadas y devuelve el archivo a su último estado estable.
- `git stash`: Guarda cambios temporales en un casillero de reserva para despejar la mesa de trabajo, recuperables con `git stash pop`.

---

6. Módulo 5: El Ecosistema GitHub — Cómo Trabajan los Equipos Reales

GitHub expande las capacidades de Git hacia la colaboración en equipo y la gestión de proyectos.

6.1 Anatomía de un Repositorio Profesional

1. `README.md`: La presentación del proyecto; expone el objetivo, arquitectura, modo de uso y autores.
2. `.gitignore`: Archivo que instruye a Git para ignorar elementos no esenciales:
3. Archivos compilados (`.class`, `.jar`).
4. Carpetas temporales del compilador (`bin/`, `build/`).
5. Credenciales privadas o configuraciones locales sensibles (`.env`).
6. `LICENSE`: Define formalmente las condiciones de uso y distribución legal del código.

6.2 Pull Requests (PR): La Revisión de Código por Pares

En las organizaciones de desarrollo profesional, **ningún cambio ingresa a la rama principal de forma directa**. Todo aporte se canaliza mediante un **Pull Request (PR)**:

```
1. [Desarrollador] ————————> Sube su rama y abre un Pull Request (PR).
2. [Equipo / Revisor] ————> Inspecciona el diff línea por línea y comenta.
3. [GitHub Actions] ————————> Compila y corre pruebas unitarias en la nube.
4. [Aprobación Final] ————> ✅ Se autoriza el Merge hacia la rama main.
```

6.3 Gestión de Proyectos: Issues, Milestones y Tableros Kanban

- **Issues (Requerimientos / Tareas):** Registro estructurado de solicitudes, mejoras o fallas detectadas en el sistema.
- **Milestones (Hitos):** Fechas clave asociadas a un conjunto de entregables (ej. "Entrega Corte 1").
- **GitHub Projects (Tableros Kanban):** Organización visual en columnas (*Por Hacer*, *En Progreso*, *Completado*) sincronizadas con la actividad del repositorio.

6.4 Automatización de Calidad: GitHub Actions (CI/CD)

Servicio de cómputo en la nube provisto por GitHub para ejecutar procesos automáticos tras cada entrega: 1. Descarga el código actualizado. 2. Compila la solución en Java 21. 3. Ejecuta la suite de pruebas unitarias. 4. Valida que el sistema mantenga su integridad antes de permitir la integración definitiva.

---

7. Módulo 6: ¿Por qué es el Corazón de la Hoja de Vida de un Ingeniero?

7.1 El fin del currículum en papel: "Proof of Work" (Prueba de Trabajo)

En el ámbito profesional de la ingeniería de software moderna, las declaraciones teóricas en un documento Word han sido reemplazadas por **evidencia empírica verificable**:

```
┌──────────────────────────────────────────────────┐
│            EVALUACIÓN DE PERFIL PROFESIONAL      │
├──────────────────────────────────────────────────┤
│ ❌ Postulación Tradicional en Papel:             │
│    "Ingeniero proactivo con conocimientos en Java."
│    -> Diagnóstico: Declaración sin evidencia real.
│                                                  │
│ ✅ Portafolio Verificable en GitHub:             │
│    • Repositorios reales con código limpio y READMEs.
│    • Historial constante de commits ordenados.   │
│    • Debates técnicos en Pull Requests y colaboración.
│    -> Diagnóstico: EVIDENCIA TÉCNICA COMPROBADA. │
└──────────────────────────────────────────────────┘
```

7.2 Qué busca un evaluador técnico en un perfil de GitHub

1. **Claridad y Disciplina en la Comunicación:** Mensajes de commit informativos y consistentes frente a descripciones ambiguas.
2. **Arquitectura y Estándares:** Separación adecuada de responsabilidades en paquetes y uso apropiado de convenciones.
3. **Constancia y Compromiso:** Gráfico de actividad periódica que refleja dedicación continua al aprendizaje y construcción de proyectos.
4. **Cultura de Colaboración:** Capacidad demostrada para debatir técnicamente en revisiones de código de manera profesional y constructiva.

8. Módulo 7: Guion de Exposición para la Clase (Estructura Pedagógica)

Estructura metodológica sugerida para una presentación magistral de **20 minutos**:

8.1 Cronograma sugerido (20 minutos)

- **[00:00 - 03:00] Apertura y Conexión:**
- Proyectar el ejemplo de la carpeta con nombres duplicados (`informe_final_este_si.docx`).
- Preguntar a la audiencia: "*¿Quién ha sentido el temor de perder su avance al intentar una mejora en un trabajo grupal?*".
- **[03:00 - 07:00] Concepto Fundamental y Analogías:**
- Explicar la diferencia entre el motor local (Git) y la plataforma en la nube (GitHub).
- Desarrollar la analogía del estudio fotográfico para ilustrar las tres zonas de trabajo.
- **[07:00 - 13:00] Ramas y Demostración Práctica:**
- Exponer el concepto del tazón de prueba / boceto paralelo para explicar las ramas.
- Proyectar el repositorio real del proyecto (como **TucanGo**), evidenciando sus ramas, historial y documentación.
- **[13:00 - 17:00] Relevancia Profesional:**
- Explicar cómo GitHub constituye la carta de presentación técnica indispensable en la industria actual.

- **[17:00 - 20:00] Espacio de Preguntas y Cierre:**
- Atender consultas de los compañeros y el docente evaluador.

8.2 Dinámicas de inicio para conectar con todo el salón

1. "*Si nos encontramos en una zona rural sin acceso a internet, ¿podemos continuar registrando versiones con Git?*"
   (Permitir participación -> Ratificar que SÍ, gracias a la naturaleza 100% distribuida de Git).
2. "*Si eliminamos accidentalmente una clase del proyecto, ¿cómo nos protege el sistema?*"
   (Explicar cómo `git restore` recupera el archivo intacto en un segundo).

8.3 Preguntas del profesor y respuestas de nivel senior

- **Pregunta 1: "¿Por qué utilizar Git en lugar de servicios de almacenamiento en la nube convencionales como Drive o Dropbox?"**
  Respuesta Senior: "*Los servicios de almacenamiento sincronizan archivos de manera global y pueden sobrescribir cambios concurrentes. Git opera a nivel de líneas de código, permite desarrollo paralelo mediante ramas independientes, preserva un registro inmutable con firmas criptográficas y ofrece herramientas de trazabilidad como `git blame`.*"

- **Pregunta 2: "¿Qué distinción técnica existe entre `git fetch` y `git pull`?"**
  Respuesta Senior: "*`git fetch` descarga las referencias y metadatos del servidor remoto al repositorio local sin modificar el espacio de trabajo activo. `git pull` ejecuta de manera consecutiva un `git fetch` seguido de un `git merge`, integrando las novedades en la rama de trabajo.*"

- **Pregunta 3: "¿Cuál es el motivo técnico para excluir carpetas de compilación como `bin/` del repositorio?"**
  Respuesta Senior: "*Los archivos compilados son resultados derivados que dependen del entorno de ejecución y saturan innecesariamente el historial. El estándar de ingeniería establece versionar exclusivamente el código fuente (`.java`), garantizando que cada entorno compile de forma limpia.*"

---

9. Módulo 8: Directorio de Enlaces y Recursos de Estudio

Recursos oficiales y formativos de referencia:

📖 Documentación Oficial y Libros (Acceso Gratuito)

- **Libro Oficial Pro Git — Scott Chacon (Español)**: Texto académico y técnico de referencia internacional.
- **Guía Oficial de Primeros Pasos en GitHub (Español)**: Manual oficial de Microsoft y GitHub.
- **Guía de Formato Markdown en GitHub**: Estándar para redacción de documentación técnica.

🎮 Plataformas Prácticas e Interactivas

- **Learn Git Branching (Simulador Visual)**: Entorno interactivo visual para practicar ramas, fusiones y navegación.
- **GitHub Skills (Laboratorios Prácticos)**: Cursos guiados en entornos reales de GitHub.

📽️ Contenido Formativo en Video (YouTube - Español)

- **Curso de Git y GitHub desde Cero — Midudev (YouTube)**: Presentación didáctica, moderna y visual.
- **Curso de Git y GitHub para Principiantes — MoureDev (YouTube)**: Explicación gradual y orientada a buenas prácticas.

- **Tutorial Completo de Git y GitHub — Fazt Code (YouTube)**: Desarrollo técnico en terminal y trabajo colaborativo.

Cátedra pedagógica compilada para el programa de Ingeniería de Sistemas — Universidad de la Amazonia.